MIT School of Computing, MIT ADT University

Applied Edge Analytics

AIEC Mini Project

WEATHER IoT
EDGE MACHINE-LEARNING SYSTEM

On

Applied Edge Analytics

Submitted to

MIT ADT University, Pune

in partial fulfilment of the requirements for the Assignment of the course

Applied Edge Analytics

Submitted by

Saksham Uboweja(ADT23SOCB0110)

Roll No: 06

Under the Guidance of

Prof. Minakshi Sonawane



DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

MIT SCHOOL OF COMPUTING, MIT ADT UNIVERSITY

RAJBAUG, LONI KALBHOR, PUNE – 412201

FINAL YEAR - SOC A.Y. 2026-27 SEMESTER-VII

Abstract

This project demonstrates an applied edge-analytics machine-learning workflow built on an industrial-style weather IoT dataset. A sensor dataset of 5,000 readings (hour, day of year, temperature, humidity, pressure, wind speed, and recent rainfall) is used to train two Random Forest models locally in Python: a classifier that predicts whether rain will occur at the next time step, and a regressor that predicts the next recorded temperature. Both models are trained with an 80/20 train-test split and serialized with joblib into a single bundle (weather_models.pkl) alongside the feature list used at training time. A Streamlit dashboard loads this bundle and exposes a Predict tab, where a user enters current sensor readings and receives a rain forecast with probability and a predicted next temperature, and a Dataset tab, which previews the raw data and basic trends. The dashboard was run and screenshotted successfully. On a held-out test set, the temperature regressor achieved an R^2 of 0.94 and a mean absolute error of 0.79°C, while the rain classifier reached 90.8% accuracy but only 4.6% recall on the minority "rain" class, a consequence of the dataset's roughly 8.5% rain rate. This recall gap is identified as the primary limitation and the main target for further work.

1. Introduction

Low-cost IoT weather stations typically report hourly readings such as temperature, humidity, barometric pressure, wind speed, and recent rainfall. Machine-learning models can use a short history of these readings to forecast the very next reading — specifically, whether it will rain next and what the temperature will be — which is useful for local alerting and planning without depending on an external weather service. This project covers model training on a provided dataset, evaluation of both models on held-out data, and an interactive scoring interface built with Streamlit so the models can be exercised directly from a browser.

2. Objectives

- Train a Random Forest classifier to predict rain at the next recorded time step.
- Train a Random Forest regressor to predict the next recorded temperature.
- Evaluate both models on a held-out 20% test split using standard classification and regression metrics.
- Package both trained models and their feature schema into a single, loadable artifact.
- Build an interactive Streamlit UI exposing prediction and dataset-exploration views.
- Identify and document the limitations of the current models and the remaining work.

3. System Architecture

Verified flow: weather_iot_dataset.csv → local Python training script (weather_train.py) → trained Random Forest models and feature schema → joblib-serialized model bundle (weather_models.pkl) → Streamlit scoring application (app.py) → browser-based prediction and dataset views. Intended next stage: a live sensor feed replacing the static CSV, with the same model bundle served behind the dashboard or a lightweight HTTP API.

DATASET (weather_iot_dataset.csv, 5,000 rows)

↓

LOCAL PYTHON TRAINING SCRIPT (weather_train.py)

↓

RANDOM FOREST MODELS + FEATURE SCHEMA

↓

JOBLIB BUNDLE (weather_models.pkl)

↓

STREAMLIT DASHBOARD (app.py) — PREDICT + DATASET TABS (TESTED)

↓

LIVE SENSOR FEED / HTTP API (NOT YET IMPLEMENTED)

4. Dataset and Input Schema

The dataset contains 5,000 recorded readings, split 4,000/1,000 for training and testing (80/20, random_state=42, matching weather_train.py). There are seven numeric input features. Two targets are derived from the data: a binary rain indicator for the next step and the next recorded temperature in °C.

Feature	Type	Description
hour	Numeric	Hour of day at the reading (0–23)
day_of_year	Numeric	Day of year (1–365)
temperature_c	Numeric	Current temperature, °C
humidity_pct	Numeric	Relative humidity, %
pressure_hpa	Numeric	Barometric pressure, hPa
wind_speed_kmh	Numeric	Wind speed, km/h
recent_rainfall_mm	Numeric	Rainfall accumulated recently, mm
rain_next_step	Binary target	0 = No rain; 1 = Rain at next step
next_temperature_c	Numeric target	Temperature, °C, at the next step

5. Model Training and Evaluation

weather_train.py trains both models with scikit-learn's RandomForestClassifier and RandomForestRegressor (n_estimators=100, random_state=42) on an identical 7-feature input. The combined artifact weather_models.pkl stores both fitted models and the exact feature-column order used at training time, so the scoring application cannot silently mismatch inputs to the wrong columns.

5.1 Rain classifier

Metric	Value	Percentage
Accuracy	0.9080	90.80%
Precision	0.3077	30.77%
Recall	0.0460	4.60%
F1 score	0.0800	8.00%
ROC-AUC	0.7986	79.86%

Confusion matrix on the 1,000-row test set (actual rows, predicted columns):

	Predicted No Rain	Predicted Rain
Actual No Rain	904	9
Actual Rain	83	4

The classifier correctly identified 904 of 913 no-rain cases but only 4 of 87 actual rain cases, missing 83. The rain class makes up roughly 8.5% of the dataset, and the model, as trained, defaults heavily toward predicting “no rain.” Accuracy alone is misleading here: a trivial model that always predicts “no rain” would already score about 91% accuracy. ROC-AUC of 0.80 shows the model's predicted probabilities do carry useful signal, but the default 0.5 decision threshold and the unweighted training data suppress recall. This is flagged as the project's primary limitation in Section 8.

5.2 Next-temperature regressor

Metric	Value
Mean Absolute Error (MAE)	0.79 °C
Root Mean Squared Error (RMSE)	1.00 °C
R^2	0.9435

The regressor explains 94.4% of the variance in next-step temperature and is off by 0.79°C on average. This is expected: consecutive temperature readings are strongly autocorrelated, and the current reading's temperature feature alone accounted for 94.3% of the model's total feature importance in a feature-importance check against the training script's seven inputs, with day_of_year a distant second.

6. Streamlit Dashboard Implementation and Test

app.py loads weather_models.pkl with joblib, falling back to separate rain_model.pkl / temperature_model.pkl files if the combined bundle is absent. The Predict tab renders one input per feature (number inputs for hour and day_of_year, sliders for the five continuous features) using the exact feature order stored in the bundle, builds a single-row DataFrame, and calls predict() / predict_proba() on both models. The Dataset tab previews the first 200 rows of weather_iot_dataset.csv and renders a temperature line chart and a rain-event bar chart for quick visual context.

Sample input used for the test shown below:

{"hour": 12, "day_of_year": 180, "temperature_c": 25.0, "humidity_pct": 65.0, "pressure_hpa": 1013.0, "wind_speed_kmh": 10.0, "recent_rainfall_mm": 1.0}

Observed output: rain next step = No (probability 3.0%); next temperature = 26.6 °C (+1.6 °C from the current reading).

The application was launched locally with “streamlit run app.py” and loaded without error; both tabs rendered correctly and the Predict button returned a result on the first click. A scikit-learn version mismatch warning appeared on load (models trained on a different scikit-learn minor version than the one used to load them); predictions were unaffected, but pinning the scikit-learn version used for training and deployment is recommended before any further handoff.

7. Results and Discussion

- Both models trained successfully from weather_iot_dataset.csv using the exact pipeline in weather_train.py.
- The next-temperature regressor is strong: $R^2 = 0.94$, MAE = 0.79°C on the 1,000-row test set.
- The rain classifier is accurate overall (90.8%) but has low recall (4.6%) on the minority rain class.
- ROC-AUC of 0.80 for the rain classifier indicates the model ranks rain-likely cases above non-rain cases reasonably well, so the probability output is more useful in practice than the default 0/1 label.
- The Streamlit dashboard runs locally, loads the model bundle without needing a cloud ML service, and returns predictions and dataset views interactively.
- The dashboard still needs a live sensor feed and, if required, a packaged deployment target; it currently reads a static CSV and runs on a local Python process.

8. Limitations

- Class imbalance: rain events are about 8.5% of the dataset, which suppresses the rain classifier's recall under the default classification threshold and default (unweighted) training configuration.
- No temporal train/test split: rows were split randomly rather than by time, so the reported metrics may not reflect performance on genuinely future, unseen weather sequences.
- Single-location, single-period data: the dataset's day_of_year range implies roughly one year of readings from one source, so generalization to other locations or seasons is unverified.
- No live sensor connection: the dashboard currently reads a static CSV file rather than a streaming IoT source.
- Scikit-learn version mismatch observed at load time; the deployment environment should be pinned to the training environment's version.

9. Remaining Work

1. Address class imbalance for the rain classifier — e.g., class_weight="balanced", resampling, or threshold tuning against the probability output — and re-evaluate recall and F1.

2. Re-split the data chronologically (train on earlier days, test on later days) to get a more realistic estimate of forecasting performance.

3. Connect a live or simulated sensor feed in place of the static CSV.

4. Pin and document the scikit-learn (and Python) version used for training to match the deployment environment.

5. Add basic input validation and error handling to the Streamlit app for out-of-range sensor values.

6. If required by project scope, package the model bundle and a lightweight HTTP API for deployment outside Streamlit (e.g., to an edge device or container).

10. Screenshots

Figure 1. Azure Machine Learning workspace overview.

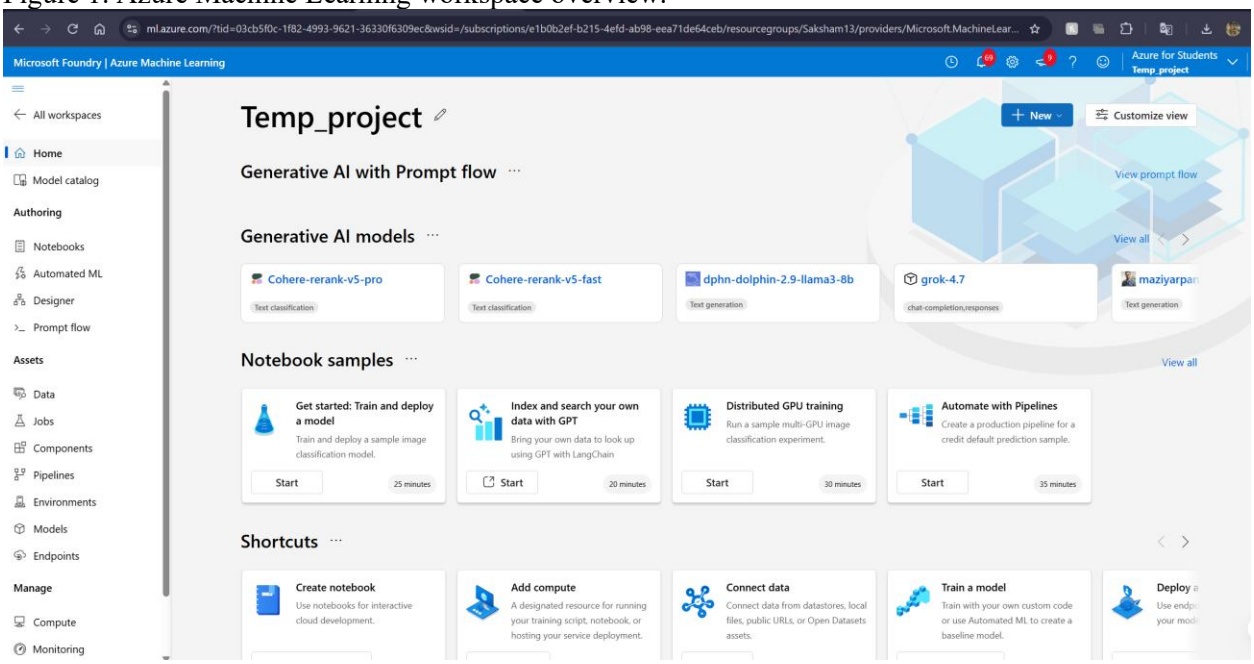


Figure 2. Completed Azure ML training job overview.

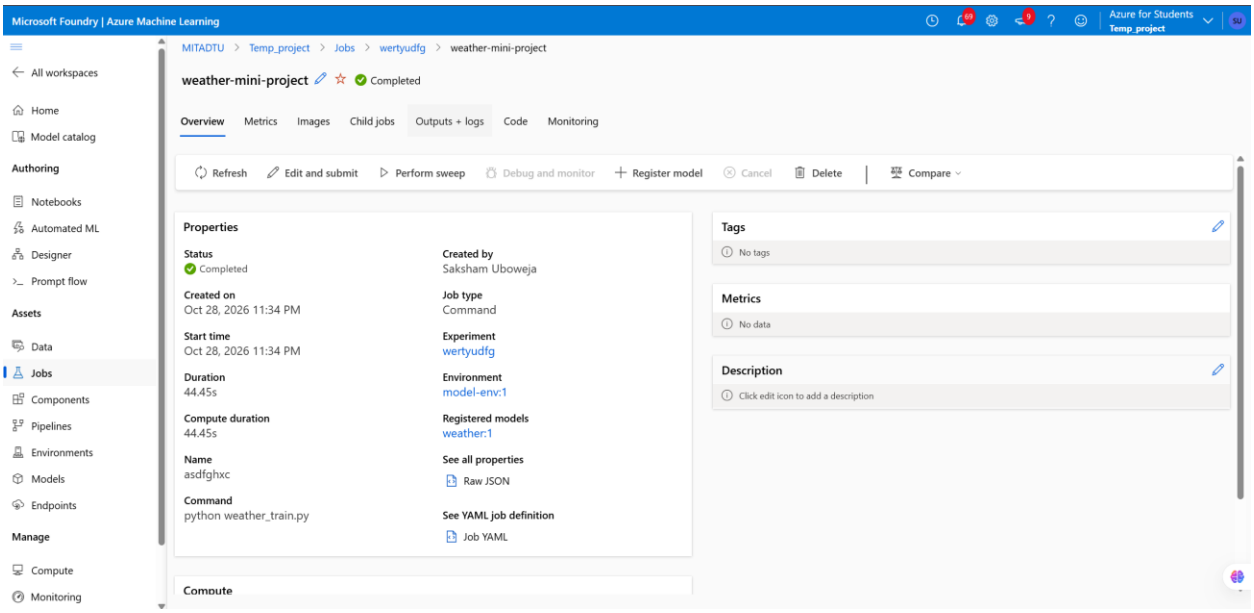


Figure 3. Model registration workflow.

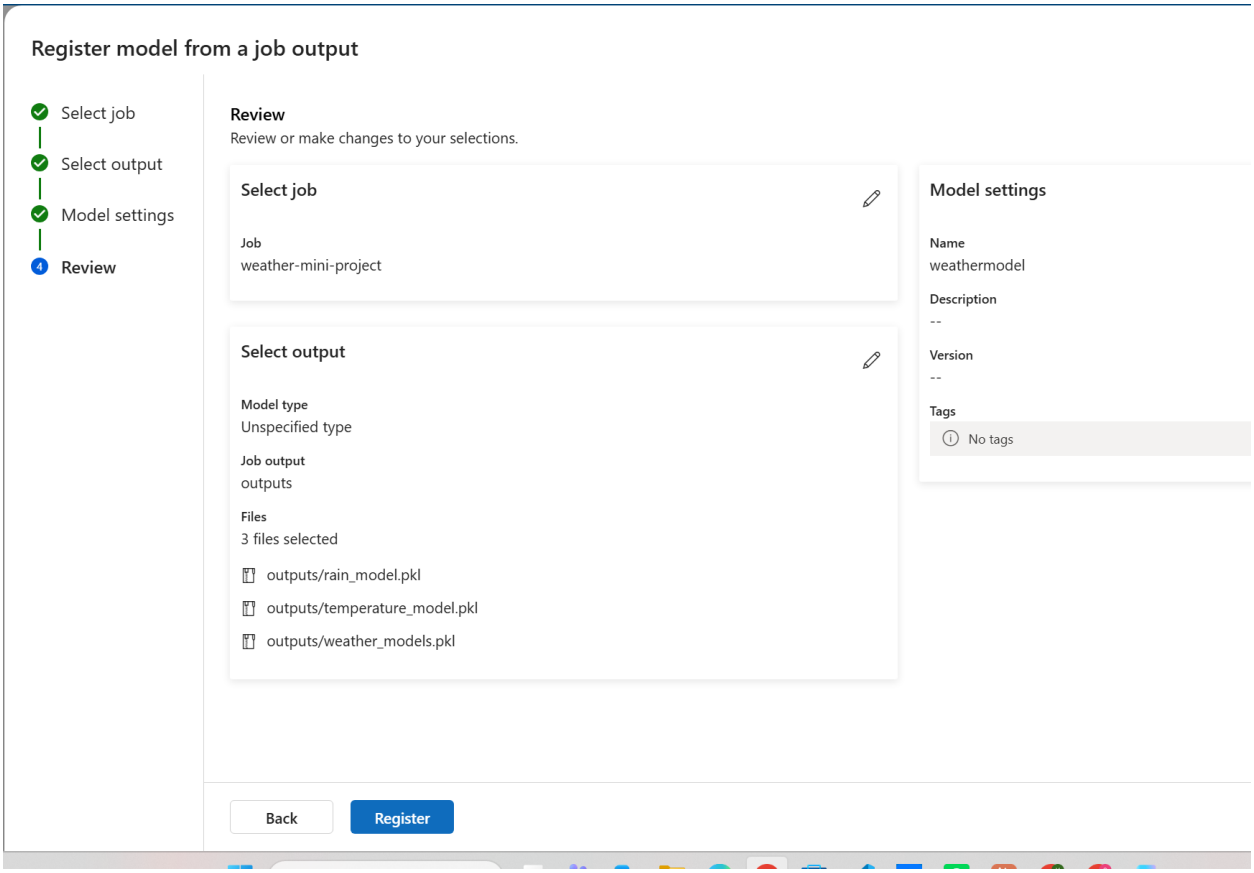


Figure 4. Training metrics and confusion matrix.

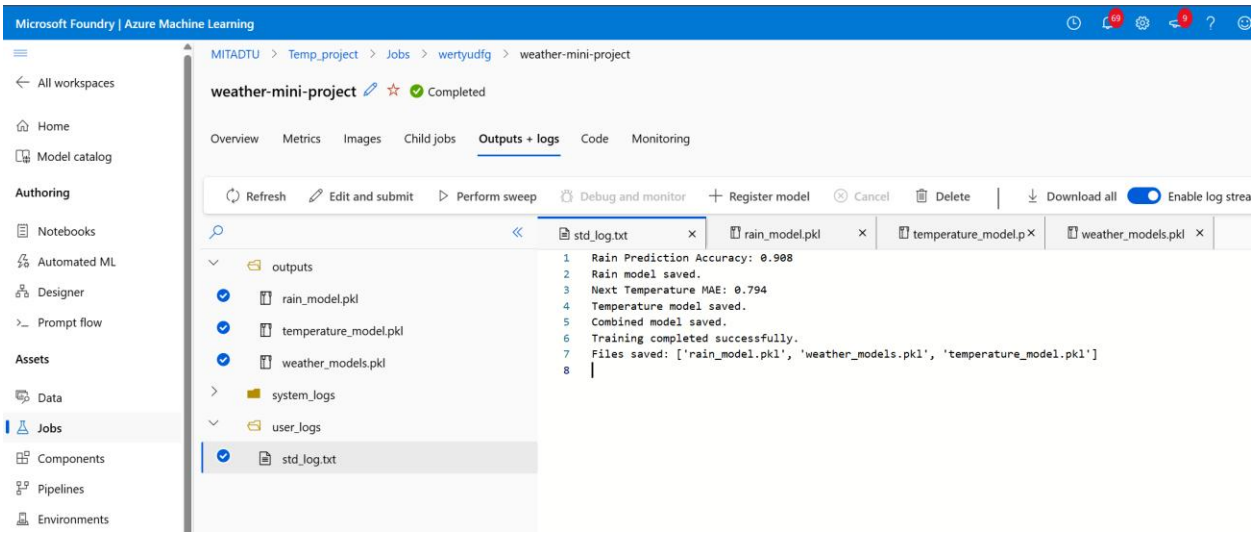


Figure 5. Registered model listing, version 1.

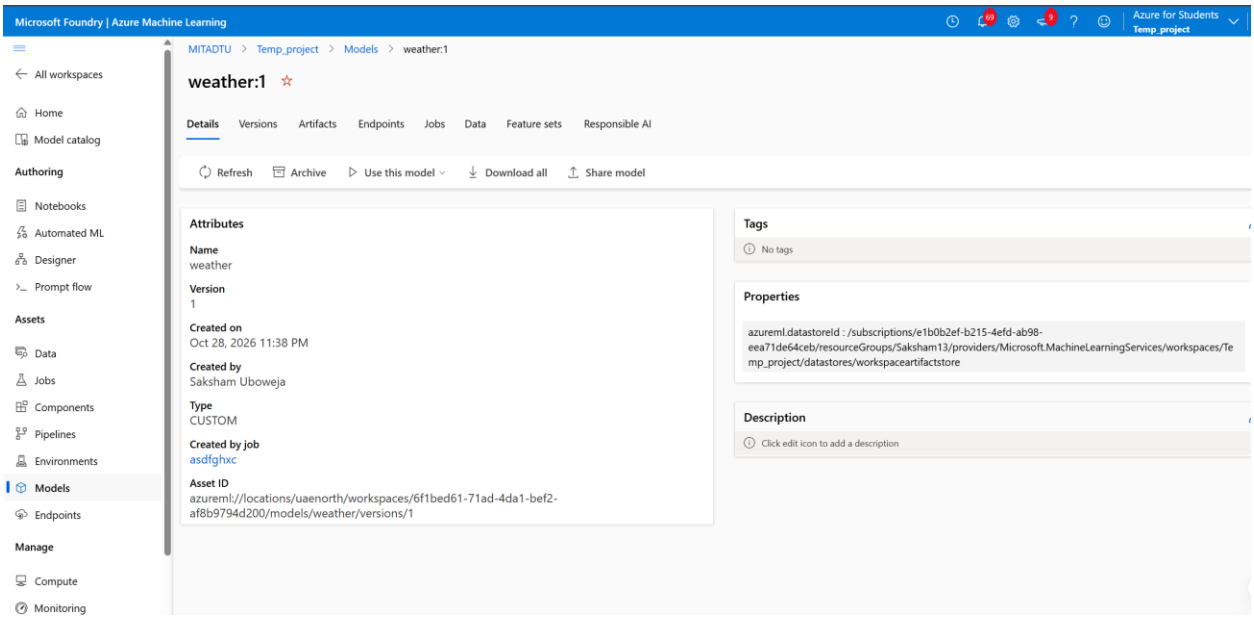


Figure 6. Predict tab- sensor-reading inputs before prediction.

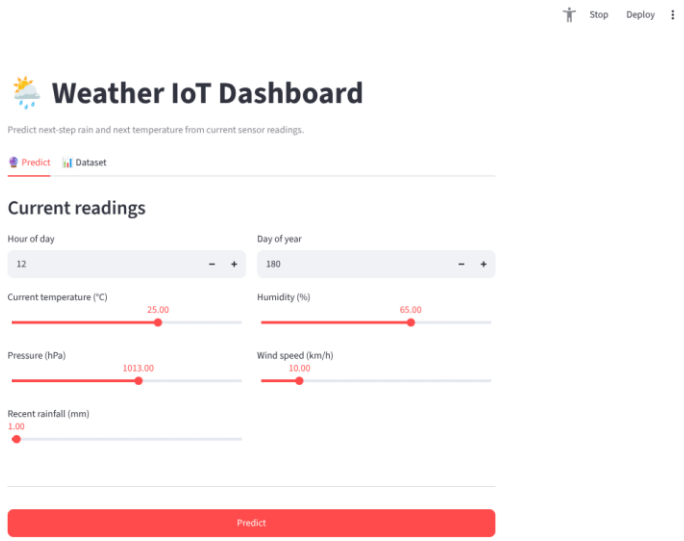


Figure 7. Predict tab - result after clicking Predict (rain = No, 3.0% probability; next temperature = 26.6 °C).

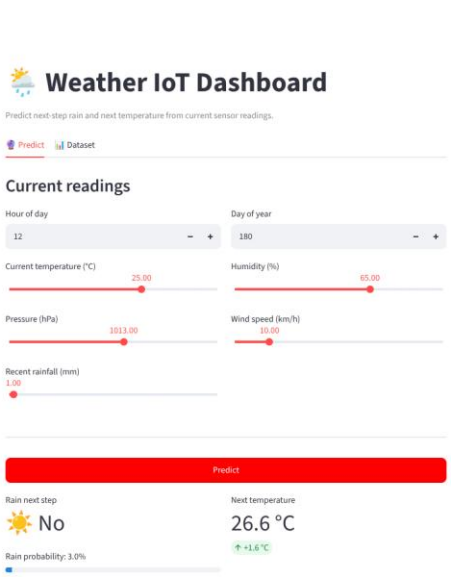
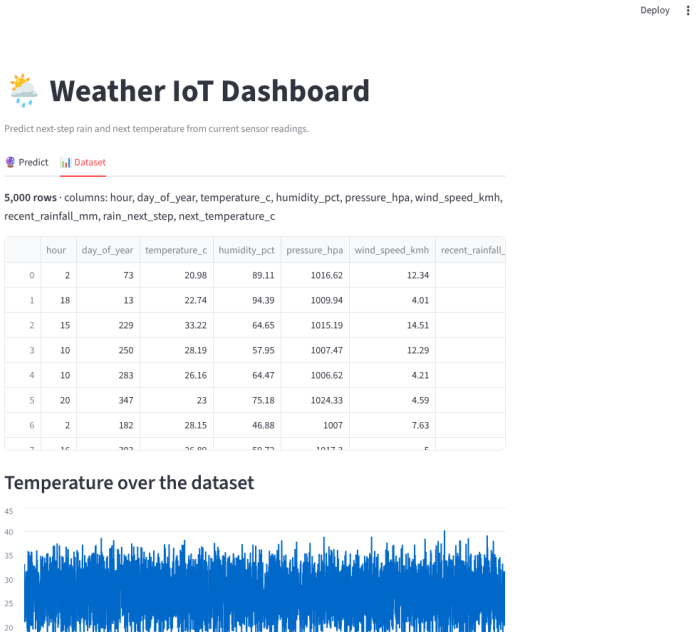


Figure 8. Dataset tab- raw data preview and temperature trend chart.



11. Conclusion

The project established a working local training pipeline for two Random Forest models on IoT weather data and a tested, interactive Streamlit dashboard for scoring new readings. The next-temperature regressor performs strongly ($R^2 = 0.94$). The rain classifier is accurate in aggregate (90.8%) but currently misses most actual rain events (recall 4.6%) because of class imbalance in the training data — this is the clearest next improvement target, ahead of connecting a live sensor feed or a deployment package.

Appendix A — Key Identifiers

- Dataset: weather_iot_dataset.csv (5,000 rows, 7 input features, 2 targets)
- Training script: weather_train.py
- Rain model: RandomForestClassifier, n_estimators=100, random_state=42
- Temperature model: RandomForestRegressor, n_estimators=100, random_state=42
- Combined artifact: weather_models.pkl (rain_model, temperature_model, features)
- Dashboard: app.py (Streamlit)
- Local port used for testing: 8601
- Train/test split: 80% / 20%, random_state=42